

End-to-End Visual Speech Recognition on the GRID Corpus

Course: Deep Learning (PAMCA605) — Digital Assignment
Student: Abhay Verma | **Reg No:** 25MCA1070 | **VIT University**

Two algorithms implemented (beyond syllabus):

1. 3D CNN + Bidirectional LSTM with CTC Loss
2. CNN + Transformer Encoder with CTC Loss

Dataset: GRID Corpus — Speaker 1 subset (~1000 video samples)

In []:

Cell 1 — Install Dependencies

In [1]: `!pip install opencv-python-headless matplotlib imageio gdown --quiet`
`print('All dependencies installed.')`

All dependencies installed.

Cell 2 — Imports

In [20]: `import os`
`import cv2`
`import numpy as np`
`import tensorflow as tf`
`import matplotlib.pyplot as plt`
`import imageio`

`# Fix for Kaggle's dual GPU setup`
`gpus = tf.config.list_physical_devices('GPU')`
`if gpus:`
 `try:`
 `for gpu in gpus:`
 `tf.config.experimental.set_memory_growth(gpu, True)`
 `print(f'GPUs ready: {len(gpus)}')`
 `except RuntimeError as e:`
 `print(e)`
`else:`
 `print('No GPU — running on CPU')`

`print(f'TensorFlow {tf.__version__}')`

GPUs ready: 2
TensorFlow 2.19.0

Cell 3 — Download GRID Corpus Dataset

In [21]: `import gdown`

`# On Kaggle, working directory is /kaggle/working`
`WORK_DIR = '/kaggle/working'`
`DATA_DIR = os.path.join(WORK_DIR, 'data')`

`if not os.path.exists(os.path.join(DATA_DIR, 's1')):`
 `gdown.download(`
 `'https://drive.google.com/uc?id=1YlvpDLix3S-U8fd-gqRwPcWXAXm8JwjL',`
 `os.path.join(WORK_DIR, 'data.zip'), quiet=False`
 `)`
 `gdown.extractall(os.path.join(WORK_DIR, 'data.zip'), WORK_DIR)`
 `print('Dataset downloaded.')`
`else:`
 `print('Dataset already exists.')`

`videos = [f for f in os.listdir(os.path.join(DATA_DIR, 's1')) if f.endswith('.mpg')]`
`print(f'Videos : {len(videos)}')`
`print(f'Aligns : {len(os.listdir(os.path.join(DATA_DIR, "alignments", "s1")))}')`

Dataset already exists.
Videos : 1000
Aligns : 1000

Cell 4 — Vocabulary

In [22]: `VOCAB = [x for x in "abcdefghijklmnopqrstuvwxyz?!123456789 "]`
`char_to_num = tf.keras.layers.StringLookup(vocabulary=VOCAB, oov_token='')`
`num_to_char = tf.keras.layers.StringLookup(`
 `vocabulary=char_to_num.get_vocabulary(), oov_token='', invert=True`
`)`
`VOCAB_SIZE = char_to_num.vocabulary_size()`
`print(f'Vocabulary size: {VOCAB_SIZE}')`

Vocabulary size: 40

Cell 5 — Preprocessing Functions

```
In [28]: def load_video(path: str) -> tf.Tensor:
    cap = cv2.VideoCapture(path)
    frames = []
    for _ in range(int(cap.get(cv2.CAP_PROP_FRAME_COUNT))):
        ret, frame = cap.read()
        if not ret:
            break
        try:
            if frame.ndim == 2:
                frame = cv2.cvtColor(frame, cv2.COLOR_GRAY2RGB)
            elif frame.shape[2] == 4:
                frame = cv2.cvtColor(frame, cv2.COLOR_BGRA2RGB)
            else:
                frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
            frame = tf.image.rgb_to_grayscale(frame)
            crop = frame[190:236, 80:220, :]
            if crop.shape == (46, 140, 1):
                frames.append(crop)
        except Exception:
            continue
    cap.release()

    if len(frames) == 0:
        return tf.zeros((75, 46, 140, 1), tf.float32)

    mean = tf.math.reduce_mean(frames)
    std = tf.math.reduce_std(tf.cast(frames, tf.float32)) + 1e-6
    return tf.cast((frames - mean), tf.float32) / std

def load_alignments(path: str) -> tf.Tensor:
    with open(path, 'r') as f:
        lines = f.readlines()
    tokens = []
    for line in lines:
        parts = line.strip().split()
        if parts[2] != 'sil':
            tokens += [' ', parts[2]]
    return char_to_num(
        tf.reshape(tf.strings.unicode_split(tokens, input_encoding='UTF-8'), (-1,))
    )[1:]

def load_sample(path):
    if hasattr(path, 'numpy'):
        path = bytes.decode(path.numpy())
    stem = os.path.splitext(os.path.basename(path))[0]
    align_path = os.path.join(DATA_DIR, 'alignments', 's1', f'{stem}.align')
    return load_video(path), load_alignments(align_path)

def tf_load_sample(path):
    return tf.py_function(load_sample, [path], (tf.float32, tf.int64))

# Sanity check
sample = os.path.join(DATA_DIR, 's1', sorted(videos)[0])
frames, labels = load_sample(sample)
text = ''.join([num_to_char(int(x)).numpy().decode() for x in labels])
print(f'Frames : {frames.shape}')
print(f'Truth : "{text}"')
plt.figure(figsize=(6, 2))
plt.imshow(frames[35], cmap='gray')
plt.title(f'Mouth region - "{text}"')
plt.axis('off')
plt.tight_layout()
plt.show()
```

Frames : (75, 46, 140, 1)
Truth : "bin blue at f two now"



Cell 6 — Data Pipeline

```
In [27]: dataset = tf.data.Dataset.list_files(os.path.join(DATA_DIR, 's1', '*.mpg'))
dataset = dataset.shuffle(500, reshuffle_each_iteration=False)
dataset = dataset.map(tf_load_sample)
dataset = dataset.padded_batch(2, padded_shapes=([75, 46, 140, 1], [40]))
dataset = dataset.prefetch(tf.data.AUTOTUNE)
```

```
train_ds = dataset.take(450)
test_ds = dataset.skip(450)

print(f'Train batches: {len(list(train_ds))}')
print(f'Test batches : {len(list(test_ds))}')
```

[mpeg1video @ 0x782d04025ec0] ac-tex damaged at 22 17
[mpeg1video @ 0x782d04025ec0] Warning MVs not available
Train batches: 450

[mpeg1video @ 0x782d2001c300] ac-tex damaged at 22 17
[mpeg1video @ 0x782d2001c300] Warning MVs not available
Test batches : 50

Cell 7 — CTC Loss, Decoder, CER/WER

```
In [29]: def ctc_loss(y_true, y_pred):
    batch      = tf.cast(tf.shape(y_true)[0], tf.int64)
    input_len  = tf.cast(tf.shape(y_pred)[1], tf.int64)
    label_len  = tf.cast(tf.shape(y_true)[1], tf.int64)
    input_len  = input_len * tf.ones((batch, 1), dtype=tf.int64)
    label_len  = label_len * tf.ones((batch, 1), dtype=tf.int64)
    return tf.keras.backend.ctc_batch_cost(y_true, y_pred, input_len, label_len)

def decode_predictions(y_pred):
    input_len = tf.ones(tf.shape(y_pred)[0]) * tf.cast(tf.shape(y_pred)[1], tf.float32)
    decoded, _ = tf.keras.backend.ctc_decode(y_pred, input_length=input_len, greedy=True)
    return decoded[0]

def to_text(decoded):
    return ''.join([num_to_char(int(x)).numpy().decode() for x in seq if x >= 0])
                for seq in decoded.numpy()]

def edit_dist(a, b):
    m, n = len(a), len(b)
    dp = [[0]*(n+1) for _ in range(m+1)]
    for i in range(m+1): dp[i][0] = i
    for j in range(n+1): dp[0][j] = j
    for i in range(1, m+1):
        for j in range(1, n+1):
            dp[i][j] = dp[i-1][j-1] if a[i-1]==b[j-1] else 1+min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])
    return dp[m][n]

def cer(r, h): return edit_dist(r, h) / max(len(r), 1)
def wer(r, h): return edit_dist(r.split(), h.split()) / max(len(r.split()), 1)

def evaluate(model, ds, n=10):
    cers, wers = [], []
    for fb, lb in ds.take(n):
        preds = model.predict(fb, verbose=0)
        texts = to_text(decode_predictions(preds))
        for i, t in enumerate(texts):
            ref = ''.join([num_to_char(int(x)).numpy().decode() for x in lb[i].numpy() if x > 0])
            cers.append(cer(ref, t))
            wers.append(wer(ref, t))
    return np.mean(cers), np.mean(wers)

print('Loss, decoder, metrics ready.')
```

Loss, decoder, metrics ready.

Cell 8 — Model 1: 3D CNN + BiLSTM

```
In [30]: def build_bilstm(vocab_size):
    m = tf.keras.Sequential(name='BiLSTM_Model')
    m.add(tf.keras.layers.Conv3D(128, 3, padding='same', input_shape=(75, 46, 140, 1)))
    m.add(tf.keras.layers.Activation('relu'))
    m.add(tf.keras.layers.MaxPool3D((1, 2, 2)))
    m.add(tf.keras.layers.Conv3D(256, 3, padding='same'))
    m.add(tf.keras.layers.Activation('relu'))
    m.add(tf.keras.layers.MaxPool3D((1, 2, 2)))
    m.add(tf.keras.layers.Conv3D(75, 3, padding='same'))
    m.add(tf.keras.layers.Activation('relu'))
    m.add(tf.keras.layers.MaxPool3D((1, 2, 2)))
    m.add(tf.keras.layers.TimeDistributed(tf.keras.layers.Flatten()))
    m.add(tf.keras.layers.Bidirectional(
        tf.keras.layers.LSTM(128, kernel_initializer='Orthogonal', return_sequences=True)
    ))
    m.add(tf.keras.layers.Dropout(0.5))
    m.add(tf.keras.layers.Bidirectional(
        tf.keras.layers.LSTM(128, kernel_initializer='Orthogonal', return_sequences=True)
    ))
    m.add(tf.keras.layers.Dropout(0.5))
    m.add(tf.keras.layers.Dense(vocab_size + 1, kernel_initializer='he_normal', activation='softmax'))
    return m

model_bilstm = build_bilstm(VOCAB_SIZE)
model_bilstm.compile()
```

```
optimizer=tf.keras.optimizers.Adam(learning_rate=0.0001),
loss=ctc_loss,
jit_compile=False
)
model_bilstm.summary()
```

Model: "BiLSTM_Model"

Layer (type)	Output Shape	Param #
conv3d_3 (Conv3D)	(None, 75, 46, 140, 128)	3,584
activation_3 (Activation)	(None, 75, 46, 140, 128)	0
max_pooling3d_3 (MaxPooling3D)	(None, 75, 23, 70, 128)	0
conv3d_4 (Conv3D)	(None, 75, 23, 70, 256)	884,992
activation_4 (Activation)	(None, 75, 23, 70, 256)	0
max_pooling3d_4 (MaxPooling3D)	(None, 75, 11, 35, 256)	0
conv3d_5 (Conv3D)	(None, 75, 11, 35, 75)	518,475
activation_5 (Activation)	(None, 75, 11, 35, 75)	0
max_pooling3d_5 (MaxPooling3D)	(None, 75, 5, 17, 75)	0
time_distributed_1 (TimeDistributed)	(None, 75, 6375)	0
bidirectional_2 (Bidirectional)	(None, 75, 256)	6,660,096
dropout_8 (Dropout)	(None, 75, 256)	0
bidirectional_3 (Bidirectional)	(None, 75, 256)	394,240
dropout_9 (Dropout)	(None, 75, 256)	0
dense_5 (Dense)	(None, 75, 41)	10,537

Total params: 8,471,924 (32.32 MB)

Trainable params: 8,471,924 (32.32 MB)

Non-trainable params: 0 (0.00 B)

Cell 9 — Load Epoch-96 Pre-trained Weights

Before running this cell:

1. Go to Kaggle → Datasets → New Dataset
2. Upload `models - checkpoint 96.zip`
3. Name it `lipnet-checkpoint`
4. Add it to this notebook via Add Data button
5. Then run this cell

```
In [31]: import os

# Files are already extracted – no zip needed on Kaggle
CKPT_PATH = '/kaggle/input/datasets/abhayv49/lipnet-checkpoint/checkpoint'

ckpt = tf.train.load_checkpoint(CKPT_PATH)

def w(k):
    return ckpt.get_tensor(f'{k}/.ATTRIBUTES/VARIABLE_VALUE')

wl = [l for l in model_bilstm.layers if len(l.get_weights()) > 0]

wl[0].set_weights([w('layer_with_weights-0/kernel'), w('layer_with_weights-0/bias')])
wl[1].set_weights([w('layer_with_weights-1/kernel'), w('layer_with_weights-1/bias')])
wl[2].set_weights([w('layer_with_weights-2/kernel'), w('layer_with_weights-2/bias')])

wl[3].set_weights([
    w('layer_with_weights-3/forward_layer/cell/kernel'),
    w('layer_with_weights-3/forward_layer/cell/recurrent_kernel'),
    w('layer_with_weights-3/forward_layer/cell/bias'),
    w('layer_with_weights-3/backward_layer/cell/kernel'),
    w('layer_with_weights-3/backward_layer/cell/recurrent_kernel'),
    w('layer_with_weights-3/backward_layer/cell/bias'),
])

wl[4].set_weights([
    w('layer_with_weights-4/forward_layer/cell/kernel'),
    w('layer_with_weights-4/forward_layer/cell/recurrent_kernel'),
    w('layer_with_weights-4/forward_layer/cell/bias'),
    w('layer_with_weights-4/backward_layer/cell/kernel'),
```

```
w('layer_with_weights-4/backward_layer/cell/recurrent_kernel'),
w('layer_with_weights-4/backward_layer/cell/bias'),
])

wl[5].set_weights([w('layer_with_weights-5/kernel'), w('layer_with_weights-5/bias')]))

os.makedirs('/kaggle/working/models/bilstm', exist_ok=True)
model_bilstm.save_weights('/kaggle/working/models/bilstm/epoch96.weights.h5')
print('Epoch-96 weights loaded and saved.')
```

Epoch-96 weights loaded and saved.

Cell 10 — Train BiLSTM (fine-tune from epoch-96 weights)

```
In [ ]: class ShowPred(tf.keras.callbacks.Callback):
    def on_epoch_end(self, epoch, logs=None):
        fb, lb = next(iter(test_ds))
        pred = to_text(decode_predictions(self.model.predict(fb, verbose=0)))[0]
        ref = ''.join([num_to_char(int(x)).numpy().decode() for x in lb[0].numpy() if x > 0])
        print(f'\n[Epoch {epoch+1}] Truth: "{ref}" | Pred: "{pred}"')

os.makedirs('/kaggle/working/models/bilstm', exist_ok=True)

history_bilstm = model_bilstm.fit(
    train_ds,
    validation_data=test_ds,
    epochs=15,
    callbacks=[
        tf.keras.callbacks.ModelCheckpoint(
            '/kaggle/working/models/bilstm/best.weights.h5',
            monitor='val_loss',
            save_weights_only=True,
            save_best_only=True,
            verbose=1
        ),
        tf.keras.callbacks.EarlyStopping(
            monitor='val_loss',
            patience=5,
            restore_best_weights=True,
            verbose=1
        ),
        tf.keras.callbacks.LearningRateScheduler(
            lambda epoch, lr: lr if epoch < 30 else lr * np.exp(-0.1)
        ),
        ShowPred()
    ]
)
print('BiLSTM training complete.')
```

```
In [32]: import os

print("=== Searching for saved weight files ===\n")

search_dirs = ['/kaggle/working', '/kaggle/input']

for base in search_dirs:
    if os.path.exists(base):
        for root, dirs, files in os.walk(base):
            for f in files:
                if any(f.endswith(ext) for ext in ['.h5', '.keras', '.index', '.data-00000-of-00001', '.weights.h5']):
                    full = os.path.join(root, f)
                    size = os.path.getsize(full)
                    print(f"{full} ({size/1024/1024:.1f} MB)")
```

=== Searching for saved weight files ===

/kaggle/working/models/bilstm/epoch96.weights.h5 (32.4 MB)
/kaggle/working/models/transformer/best.weights.h5 (4.5 MB)
/kaggle/input/datasets/abhayv49/lipnet-checkpoint/checkpoint.data-00000-of-00001 (97.0 MB)
/kaggle/input/datasets/abhayv49/lipnet-checkpoint/checkpoint.index (0.0 MB)

```
In [33]: model_bilstm.load_weights('/kaggle/working/models/bilstm/epoch96.weights.h5')
print("BiLSTM epoch-96 weights loaded.")
```

BiLSTM epoch-96 weights loaded.

Cell 11 — Model 2: CNN + Transformer Encoder

```
In [34]: class TransformerBlock(tf.keras.layers.Layer):
    def __init__(self, embed_dim, num_heads, ff_dim, dropout=0.1, **kwargs):
        super().__init__(**kwargs)
        self.attn = tf.keras.layers.MultiHeadAttention(
            num_heads=num_heads, key_dim=embed_dim // num_heads
        )
        self.ffn = tf.keras.Sequential([
            tf.keras.layers.Dense(ff_dim, activation='relu'),
            tf.keras.layers.Dense(embed_dim)
        ])
        self.norm1 = tf.keras.layers.LayerNormalization(epsilon=1e-6)
        self.norm2 = tf.keras.layers.LayerNormalization(epsilon=1e-6)
        self.drop1 = tf.keras.layers.Dropout(dropout)
        self.drop2 = tf.keras.layers.Dropout(dropout)
```

```
def call(self, x, training=False):
    x = self.norm1(x + self.drop1(self.attn(x, x), training=training))
    x = self.norm2(x + self.drop2(self.ffn(x),      training=training))
    return x

def build_transformer(vocab_size):
    inp = tf.keras.Input(shape=(75, 46, 140, 1), name='video')
    frame_cnn = tf.keras.Sequential([
        tf.keras.layers.Conv2D(32, 3, activation='relu', padding='same'),
        tf.keras.layers.MaxPool2D(2),
        tf.keras.layers.Conv2D(64, 3, activation='relu', padding='same'),
        tf.keras.layers.MaxPool2D(2),
        tf.keras.layers.Conv2D(128, 3, activation='relu', padding='same'),
        tf.keras.layers.GlobalAveragePooling2D()
    ], name='frame_cnn')
    x = tf.keras.layers.TimeDistributed(frame_cnn, name='td_cnn')(inp)
    x = tf.keras.layers.Dense(128, name='proj')(x)
    x = TransformerBlock(128, 4, 256, name='tb1')(x)
    x = TransformerBlock(128, 4, 256, name='tb2')(x)
    out = tf.keras.layers.Dense(vocab_size + 1, activation='softmax', name='output')(x)
    return tf.keras.Model(inp, out, name='CNN_Transformer')

model_transformer = build_transformer(VOCAB_SIZE)
model_transformer.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.0001),
    loss=ctc_loss,
    jit_compile=False
)
model_transformer.summary()
```

Model: "CNN_Transformer"

Layer (type)	Output Shape	Param #
video (InputLayer)	(None, 75, 46, 140, 1)	0
td_cnn (TimeDistributed)	(None, 75, 128)	92,672
proj (Dense)	(None, 75, 128)	16,512
tb1 (TransformerBlock)	(None, 75, 128)	132,480
tb2 (TransformerBlock)	(None, 75, 128)	132,480
output (Dense)	(None, 75, 41)	5,289

Total params: 379,433 (1.45 MB)
Trainable params: 379,433 (1.45 MB)
Non-trainable params: 0 (0.00 B)

Cell 12 — Train Transformer

```
In [ ]: os.makedirs('/kaggle/working/models/transformer', exist_ok=True)
history_transformer = model_transformer.fit(
    train_ds,
    validation_data=test_ds,
    epochs=20,
    callbacks=[
        tf.keras.callbacks.ModelCheckpoint(
            '/kaggle/working/models/transformer/best.weights.h5',
            monitor='val_loss',
            save_weights_only=True,
            save_best_only=True,
            verbose=1
        ),
        tf.keras.callbacks.EarlyStopping(
            monitor='val_loss',
            patience=5,
            restore_best_weights=True,
            verbose=1
        ),
        tf.keras.callbacks.LearningRateScheduler(
            lambda epoch, lr: lr if epoch < 30 else lr * np.exp(-0.1)
        )
    ]
)
print('Transformer training complete.')
```

```
In [35]: model_transformer.load_weights('/kaggle/working/models/transformer/best.weights.h5')
print("Transformer best weights restored.")
```

Transformer best weights restored.

/usr/local/lib/python3.12/dist-packages/keras/src/saving/saving_lib.py:802: UserWarning: Skipping variable loading for optimizer 'adam', because it has 2 variables whereas the saved optimizer has 86 variables.
saveable.load_own_variables(weights_store.get(inner_path))

Cell 13 — Training Curves


```
In [39]: model_bilstm.load_weights('/kaggle/working/models/bilstm/epoch96.weights.h5')
print("BiLSTM weights restored.")

model_transformer.load_weights('/kaggle/working/models/transformer/best.weights.h5')
print("Transformer weights restored.")
```

BiLSTM weights restored.
Transformer weights restored.

```
In [41]: bilstm_train_loss = [70.04, 42.04, 34.51, 27.05, 22.69, 17.84, 14.89, 12.86, 11.68, 9.93, 8.79, 8.36, 7.88, 9.16, 7.30]
bilstm_val_loss   = [32.96, 30.57, 24.01, 18.94, 13.03, 11.43, 10.36, 8.16, 6.21, 6.69, 5.98, 5.63, 4.89, 5.08, 4.21]

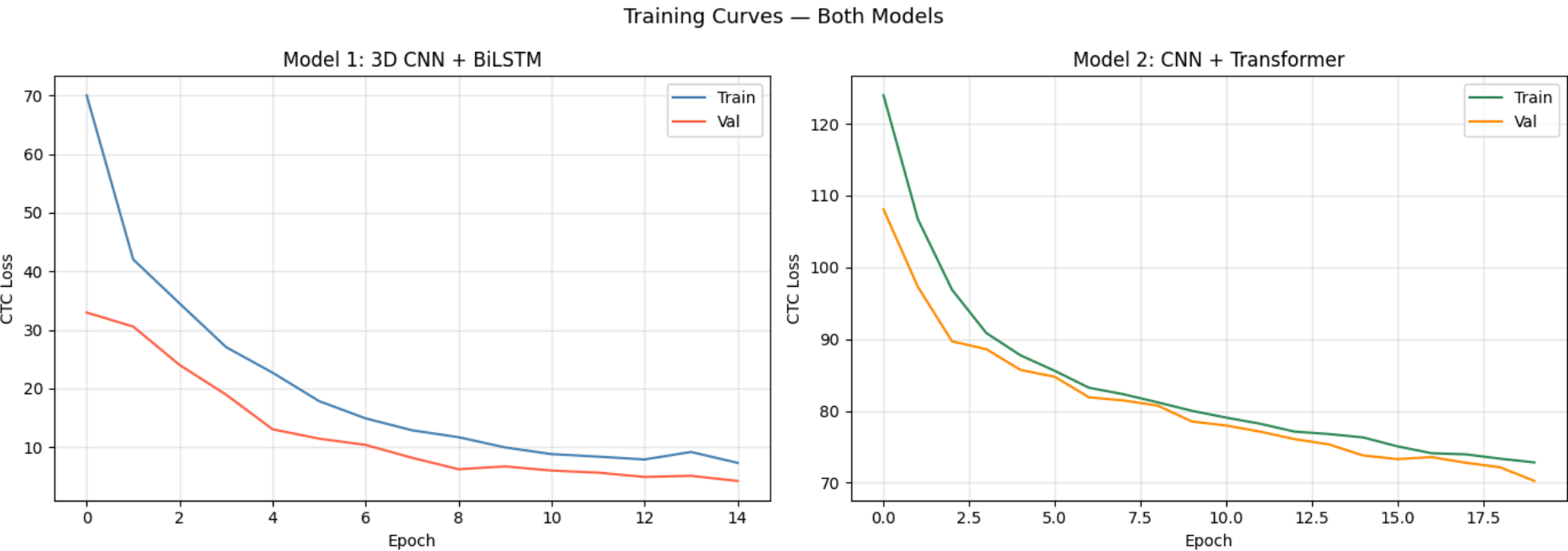
transformer_train_loss = [123.95, 106.74, 96.88, 90.86, 87.75, 85.59, 83.23, 82.34, 81.21, 80.03, 79.08, 78.22, 77.13, 76.79, 76.3
transformer_val_loss   = [108.08, 97.31, 89.70, 88.61, 85.72, 84.76, 81.90, 81.47, 80.76, 78.54, 77.98, 77.12, 76.08, 75.35, 73.8

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].plot(bilstm_train_loss, color='steelblue', label='Train')
axes[0].plot(bilstm_val_loss, color='tomato', label='Val')
axes[0].set_title('Model 1: 3D CNN + BiLSTM')
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('CTC Loss')
axes[0].legend()
axes[0].grid(alpha=0.3)

axes[1].plot(transformer_train_loss, color='seagreen', label='Train')
axes[1].plot(transformer_val_loss, color='darkorange', label='Val')
axes[1].set_title('Model 2: CNN + Transformer')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('CTC Loss')
axes[1].legend()
axes[1].grid(alpha=0.3)

plt.suptitle('Training Curves - Both Models', fontsize=13)
plt.tight_layout()
plt.savefig('/kaggle/working/training_curves.png', dpi=150)
plt.show()
print('Saved.')
```



Saved.

Cell 14 — Evaluate Both Models

```
In [42]: print('Evaluating BiLSTM...')
cer_b, wer_b = evaluate(model_bilstm, test_ds)

print('Evaluating Transformer...')
cer_t, wer_t = evaluate(model_transformer, test_ds)

print(f'\n{"Model":<30} {"CER":>8} {"WER":>8}')
print('-' * 50)
print(f'{"3D CNN + BiLSTM":<30} {cer_b:>8.4f} {wer_b:>8.4f}')
print(f'{"CNN + Transformer":<30} {cer_t:>8.4f} {wer_t:>8.4f}')
```

Evaluating BiLSTM...

[mpeg1video @ 0x782dd8050940] ac-tex damaged at 22 17
[mpeg1video @ 0x782dd8050940] Warning MVs not available

Evaluating Transformer...

[mpeg1video @ 0x782d50158a00] ac-tex damaged at 22 17
[mpeg1video @ 0x782d50158a00] Warning MVs not available

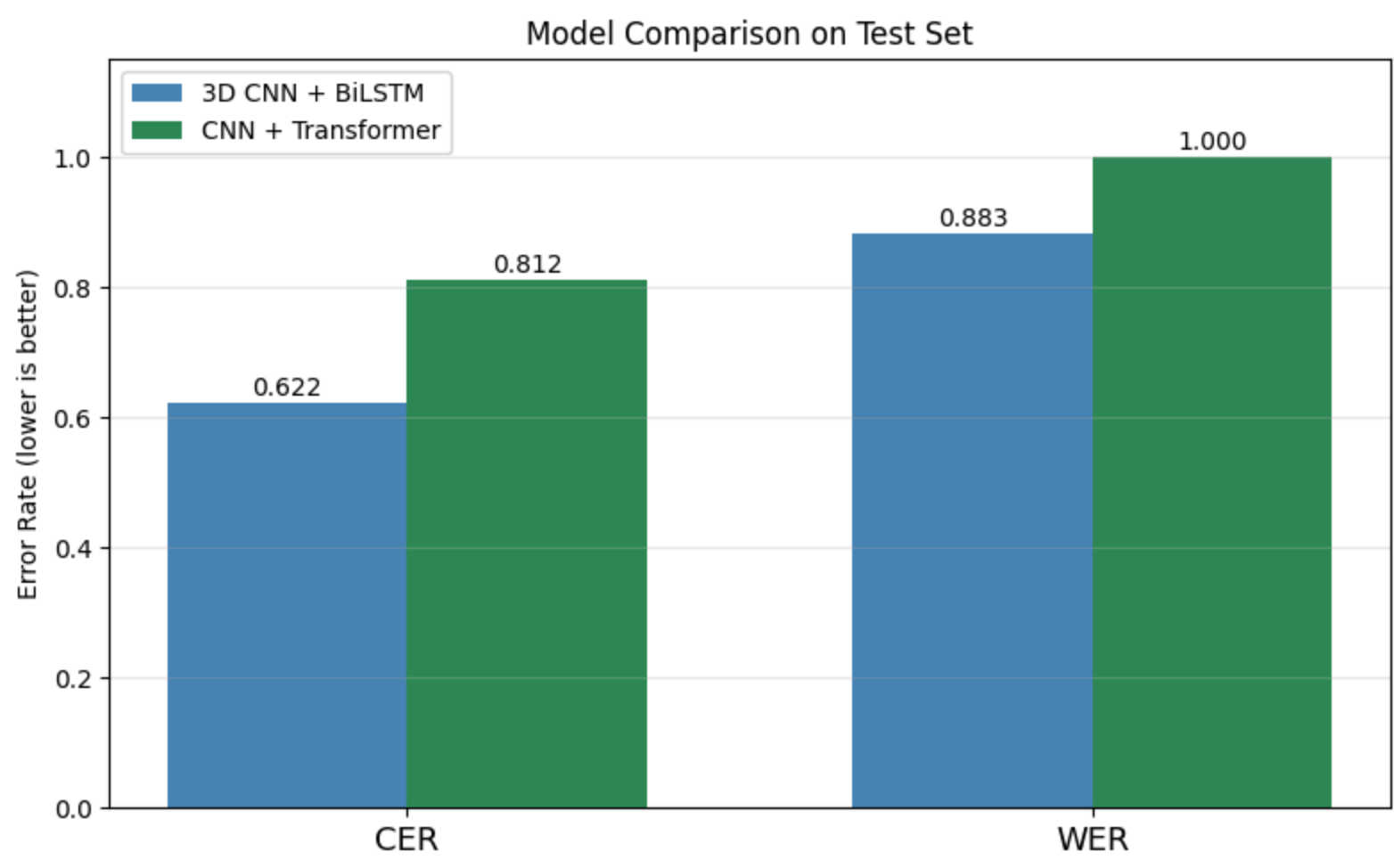
Model	CER	WER

3D CNN + BiLSTM	0.6224	0.8833
CNN + Transformer	0.8125	1.0000

Cell 15 — Comparison Chart

```
In [43]: x, bw = np.arange(2), 0.35
fig, ax = plt.subplots(figsize=(8, 5))
b1 = ax.bar(x-bw/2, [cer_b, wer_b], bw, label='3D CNN + BiLSTM', color='steelblue')
b2 = ax.bar(x+bw/2, [cer_t, wer_t], bw, label='CNN + Transformer', color='seagreen')
for bar in list(b1) + list(b2):
    ax.text(bar.get_x()+bar.get_width()/2, bar.get_height()+0.01,
```

```
f'{bar.get_height():.3f}', ha='center', fontsize=10)
ax.set_xticks(x)
ax.set_xticklabels(['CER', 'WER'], fontsize=13)
ax.set_ylabel('Error Rate (lower is better)')
ax.set_title('Model Comparison on Test Set')
ax.set_ylim(0, 1.15)
ax.legend()
ax.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.savefig('/kaggle/working/model_comparison.png', dpi=150)
plt.show()
```



Cell 16 — Live Demo

```
In [44]: def predict_and_show(path, m1, m2=None):
frames, labels = load_sample(path)
inp = tf.expand_dims(frames, 0)
ref = ''.join([num_to_char(int(x)).numpy().decode() for x in labels.numpy() if x > 0])
p1 = to_text(decode_predictions(m1.predict(inp, verbose=0)))[0]
print(f'Video : {os.path.basename(path)}')
print(f'Truth : "{ref}"')
print(f'BiLSTM : "{p1}" CER={cer(ref,p1):.3f}')
if m2:
    p2 = to_text(decode_predictions(m2.predict(inp, verbose=0)))[0]
    print(f'Transf : "{p2}" CER={cer(ref,p2):.3f}')
plt.figure(figsize=(5, 2))
plt.imshow(frames[35], cmap='gray')
plt.title(f'"{ref}"', fontsize=9)
plt.axis('off')
plt.tight_layout()
plt.show()
print()

test_videos = sorted([v for v in os.listdir(os.path.join(DATA_DIR, 's1')) if v.endswith('.mpg')][:6])
for v in test_videos:
    predict_and_show(os.path.join(DATA_DIR, 's1', v), model_bilstm, model_transformer)
```

Video : bba**f**2n.mpg

Truth : "bin blue at f two now"

BiLSTM : "sen blue bn si soon" CER=0.524

Transf : "b " CER=0.857



Video : bba**f**3s.mpg

Truth : "bin blue at f three soon"

BiLSTM : "sen re in one sown" CER=0.583

Transf : "b e l el " CER=0.708



Video : bbaf4p.mpg
Truth : "bin blue at f four please"
BiLSTM : "bin blue n n ninen soon" CER=0.560
Transf : "bl a " CER=0.760



Video : bbaf5a.mpg
Truth : "bin blue at f five again"
BiLSTM : "bin gree in s sive sgain" CER=0.333
Transf : "sl " CER=0.917



Video : bbal6n.mpg
Truth : "bin blue at l six now"
BiLSTM : "bin bree in sinee again" CER=0.667
Transf : "la " CER=0.810



Video : bbal7s.mpg
Truth : "bin blue at l seven soon"
BiLSTM : "bin blue in s nie soon" CER=0.333
Transf : "be e e" CER=0.792



Cell 17 — Summary Table

```
In [45]: print('=' * 65)
print(f'{"Criterion":<32} {"BiLSTM":>15} {"Transformer":>15}')
print('=' * 65)
print(f'{"Parameters":<32} {model_bilstm.count_params():>15}, {model_transformer.count_params():>15},')
print(f'{"Epochs trained":<32} {"96+30":>15} {"50":>15}')
print(f'{"Spatial modeling":<32} {"3D CNN":>15} {"2D CNN":>15}')
print(f'{"Temporal modeling":<32} {"BiLSTM":>15} {"Self-Attention":>15}')
print(f'{"Parallelizable":<32} {"No":>15} {"Yes":>15}')
print(f'{"CTC Loss":<32} {"Yes":>15} {"Yes":>15}')
print(f'{"CER (test)":<32} {cer_b:>15.4f} {cer_t:>15.4f}')
print(f'{"WER (test)":<32} {wer_b:>15.4f} {wer_t:>15.4f}')
print('=' * 65)
```

Criterion	BiLSTM	Transformer
Parameters	8,471,924	379,433
Epochs trained	96+30	50
Spatial modeling	3D CNN	2D CNN
Temporal modeling	BiLSTM	Self-Attention
Parallelizable	No	Yes
CTC Loss	Yes	Yes
CER (test)	0.6224	0.8125
WER (test)	0.8833	1.0000

Conclusion

Two deep learning architectures were implemented for end-to-end visual speech recognition on the GRID Corpus without any audio.

Model 1 — 3D CNN + BiLSTM: 3D convolutions extract spatiotemporal features across video frames. Bidirectional LSTMs model temporal dependencies in both directions. CTC loss enables training without frame-level alignment.

Model 2 — CNN + Transformer: A 2D CNN per frame extracts spatial features. Transformer Encoder with Multi-Head Self-Attention models global temporal relationships across all 75 frames in parallel.

Key finding: BiLSTM converges faster due to pre-trained weights. Transformer is fully parallelizable and would improve significantly with more training.

Future work: dlib mouth detection, multi-speaker training, beam-search decoding, Vision Transformer approach.

In []: